

Interface upto JDK 1.7V :

An interface is a keyword in java which is similar to a class which defines "working functionality of a class".

Upto JDK 1.7 an interface contains only abstract methods that means there is a guarantee that inside an interface we don't have concrete or general or instance methods.

From java 8 onwards we have a facility to write default and static methods.

By using interface we can achieve 100% abstraction concept because it contains only abstract methods.

In order to implement the member of an interface, java software people has provided implements keyword.

All the methods declared inside an interface is by default public and abstract so at the time of overriding we can't reduce the accessibility level.

All the variables declared inside an interface is by default public, static and final.

We should override all the abstract methods of interface to the sub classes otherwise the sub class will become as an abstract class hence object can't be created.

We can't create an object for interface, but reference can be created.

By using interface we can achieve multiple inheritance in java.
[Don't say interface came in java to solve multiple inheritance problem]

***We can achieve loose coupling using interface.

Note :- Inside an interface, We cannot write static block, non static block, constructor, public concrete method and instance variable(No Object properties)

We can say an interface defines "WHAT TO DO" on the other hand its implementer classes defines "HOW TO DO".

```
//Programs

package com.ravi.interface_demo;

sealed interface Moveable permits Car
{
    void move(); //public + abstract
    int SPEED = 100; //public + static + final
}

non-sealed class Car implements Moveable
{
    @Override
    public void move()
    {
        //SPEED = 120;   Invalid
        IO.println("Car speed is :"+SPEED);
    }
}

public class InterfaceDemo
{
    public static void main(String[] args)
    {
        Moveable m = new Car();
        m.move();
        IO.println("All Moveable objects speed :"+Moveable.SPEED);
    }
}

package com.ravi.interface_demo;

interface Bank
{
    double getCurrentBalance();
    void deposit(double amount);
    void withdraw(double amount);
}

class Customer implements Bank
{
    private double balance;

    public Customer(double balance)
    {
        this.balance = balance;
    }

    @Override
    public double getCurrentBalance()
    {
        return this.balance;
    }

    @Override
    public void deposit(double amount)
    {
        if(amount <=0)
        {
            System.err.println("Deposit amount cannot be zero OR -ve");
            return;
        }
        this.balance += amount;
        IO.println("Current balance after deposit :"+this.getCurrentBalance());
    }

    @Override
    public void withdraw(double amount)
    {
        if(amount > this.balance)
        {
            System.err.println("Insufficient Balance");
            return;
        }
        this.balance -= amount;
        IO.println("Current balance after withdraw :"+this.getCurrentBalance());
    }
}

public class InterfaceExample {

    public static void main(String[] args)
    {
        Bank bank = new Customer(10000);
        IO.println("Current balance is :"+bank.getCurrentBalance());
        bank.deposit(5000);
        bank.withdraw(3000);
    }
}
```

interface with Anonymous inner class implementation :

```
package com.ravi.interface_demo;

interface SIM
{
    void call();
    void sms();
}

public class InterfaceWithAnonymous
{
    public static void main(String[] args)
    {
        //Anonymous inner class
        SIM bsnl = new SIM()
        {
            static
            {
                IO.println("BSNL SIM");
            }

            @Override
            public void call()
            {
                IO.println("Making a call through BSNL");
            }

            @Override
            public void sms()
            {
                IO.println("Making a sms through BSNL");
            }
        };

        SIM jio = new SIM()
        {
            static
            {
                IO.println("JIO SIM");
            }

            @Override
            public void call()
            {
                IO.println("Making a call through JIO");
            }

            @Override
            public void sms()
            {
                IO.println("Making a sms through JIO");
            }
        };

        bsnl.call();
        bsnl.sms();
        jio.call();
        jio.sms();
    }
}
```

How to achieve loose coupling using interface :

Program on loose coupling : (Industry Standard Program)

Loose Coupling :- If the degree of dependency from one class object to another class is very low then it is called loose coupling. [interface is reqd for Runtime Polymorphism]

Tightly coupled :- If the degree of dependency of one class to another class is very high then it is called Tightly coupled. [Not recommended]

According to IT industry standard, we should always prefer loose coupling so the maintenance of the project will become easy.

High Cohesion [Encapsulation]:

Our private data must be accessible via public methods (setter and getters) so, in between data and method we must have high cohesion.

(tight coupling) so, validation of outer data is possible.

```
package com.ravi.loose_coupling;

public interface HotDrink
{
    void prepare();
}

package com.ravi.loose_coupling;

public class Tea implements HotDrink
{
    @Override
    public void prepare()
    {
        IO.println("Preapring Irani Tea");
    }
}

package com.ravi.loose_coupling;

public class Coffee implements HotDrink
{
    @Override
    public void prepare()
    {
        IO.println("Preparing Java Coffee");
    }
}

package com.ravi.loose_coupling;

public class Horlicks implements HotDrink
{
    @Override
    public void prepare()
    {
        IO.println("Preparing Horlicks");
    }
}

package com.ravi.loose_coupling;

public class Restaurant
{
    public static void acceptAndPrepareTheOrder(HotDrink hd)
    {
        hd.prepare();
    }
}

package com.ravi.loose_coupling;

public class LooseCouplingDemo {

    public static void main(String[] args)
    {
        Restaurant.acceptAndPrepareTheOrder(new Tea());
        Restaurant.acceptAndPrepareTheOrder(new Coffee());
        Restaurant.acceptAndPrepareTheOrder(new Horlicks());
    }
}
```

Method return type as interafce (Factory Design Pattern)